

CI/CD AUTOMATION FRAMEWORK PROPOSAL

Standardized Templates, Shift-Left Enforcement, and Event-Driven Auto-Remediation

Prepared for Engineering Leadership | Platform Engineering | For Decision

Executive Summary

Feature development is halted because our delivery pipeline can no longer be trusted. Every team hand-authors its own pipeline definition, so a pattern proven in one repository carries no assurance in the next; malformed YAML routinely reaches the main branch and fails builds long after the mistake was introduced; and recurring out-of-memory pod terminations pull the on-call team into manual recovery on nights and weekends. This proposal sets out a framework built on three pillars: a versioned shared YAML template library that makes the correct pipeline the easiest one to write; a shift-left enforcement layer pairing Spectral rulesets with pre-commit hooks so malformed definitions are rejected on the engineer's machine rather than in the build queue; and an event-driven auto-remediation service on Azure Functions and Event Grid that resolves known production failures without waking anyone. Together these return an estimated 42 operations hours per month, cut onboarding to under one working day, and move all four DORA metrics into a defensible position within a quarter.

1. Shared YAML Template Library

The library is a single governed repository, platform-pipelines, consumed by every product repository through a pinned Git reference. It is organised in three tiers so reuse happens at the right granularity rather than through copy-paste.

- **Step templates** hold the smallest reusable units — restore cache, authenticate to the registry, publish test results — encapsulating the details engineers most often get wrong.
- **Job templates** compose steps into a unit of work with a typed contract: build-and-scan, integration-test, deploy-to-environment. Parameters declare allowed values, so an invalid environment name fails at compile time.
- **Stage templates** assemble the golden path — Build, Test, Security Scan, Package, Deploy (Dev), Deploy (Staging), Deploy (Production) — with approval gates and environment checks already wired in.

A consuming pipeline therefore declares intent rather than mechanism. A service team writes roughly twenty-five lines: a repository resource pinned to a released tag, the stage template extension, and a parameter block naming the service, its runtime, and its target environments. Four hundred lines of scaffolding move behind a single reference.

The library is versioned semantically and released by tag. Consumers pin to a major version, inherit minor and patch updates automatically, and receive a two-sprint deprecation window before any breaking change. This gives the platform team a real lever: a new security control or corrected deployment pattern is rolled out centrally and inherited everywhere rather than negotiated repository by repository. Lead time and deployment frequency improve directly, because modifying a pipeline collapses from days of bespoke authoring to a parameter change.

2. Enforcement Strategy: Spectral and Pre-Commit Hooks

Templates make the correct pipeline easy to write; enforcement makes the incorrect one impossible to commit. Spectral evaluates structured documents against a declarative ruleset using JSONPath expressions, which suits pipeline YAML well. Our ruleset, `.spectral.yaml`, is versioned alongside the template library and covers four rule families:

- **Structural:** every pipeline must extend an approved stage template and must not redefine the stage sequence.
- **Security:** no hardcoded secrets, no unpinned template references, no unapproved agent pools or container images.
- **Resource:** every container specification must declare memory and CPU requests and limits — the control that directly attacks our out-of-memory incident class.
- **Convention:** naming standards, mandatory ownership metadata, and inline script blocks capped in length to force logic into version-controlled scripts.

Rules carry severity levels: errors block, warnings inform and are reviewed monthly for promotion. That lets us launch in warn-only mode for two sprints, measure the real violation profile, and tune away false positives before anything blocks a developer.

Delivery is through the pre-commit framework. A single `.pre-commit-config.yaml`, scaffolded into every repository, registers two hooks: `yamllint` for syntax and Spectral for semantic policy. On git commit they run only against staged YAML and return in under five seconds, so an engineer sees the exact file, line, and rule that failed while the change is still in working memory.

Because local hooks can be bypassed with `--no-verify`, the identical ruleset runs again as a required pull request check. This is deliberate defence in depth: the hook is the fast feedback developers value, the server-side check is the guarantee leadership relies on, and because one ruleset file drives both they cannot drift. That is what lets us claim malformed commits are eliminated, not merely reduced.

3. Event-Driven Auto-Remediation Architecture

The remediation service is designed as a five-stage pipeline — detect, normalise, decide, act, record — rather than a script bolted onto an alert. Each stage owns one responsibility, which is what makes the system safe to extend to new failure classes later.

Detect. Azure Monitor container insights watches AKS for OOMKilled terminations and sustained memory saturation, and a matching alert rule publishes a structured event to a custom Event Grid topic. Event Grid is chosen over a direct webhook because it decouples producers from handlers, retries with exponential back-off, and dead-letters undeliverable events — so a handler outage becomes a delayed response, not a lost incident.

Normalise. An Azure Function subscribed to the topic translates the payload into a canonical incident record: cluster, namespace, workload, failure class, current resource specification, and a correlation key derived from workload identity. Normalising at the boundary means the decision logic never learns Azure Monitor's schema, and a second detection source can be added without touching it.

Decide. The decision engine evaluates the incident against a policy set held in configuration, not code. For an OOM event it inspects remediation history: if the workload has been remediated fewer than three times in twenty-four hours and its memory limit sits below a pre-approved ceiling, it authorises an increase. If that budget is exhausted, the pattern indicates a leak rather than under-provisioning, so the circuit breaker trips and escalates to PagerDuty with full history attached.

Act. Actions execute through a managed identity scoped by RBAC to the specific namespaces and verbs required, and are idempotent on the correlation key so duplicate events cannot compound a change. Critically, the runtime patch is first aid only: the function simultaneously opens a pull request proposing the corrected specification in the service's manifest repository, so the fix converges back into version control.

Record. Every decision, including refusals, is written to Log Analytics with its input, policy evaluation, and outcome, and posted to the owning team's channel. This audit trail is what makes autonomous action acceptable to auditors and to workload owners.

4. ROI Projection

The model uses the trailing ninety days of incident and build telemetry. Operations currently handles roughly sixty OOM-related pod failures per month at thirty-eight minutes each across triage, intervention, verification, and closure. Eighty-five per cent are straightforward under-provisioning the policy engine can resolve autonomously.

Operational Workstream	Current (hrs/mo)	Post-Framework	Hours Saved
OOM pod recovery (60 events × 38 min)	38.0	5.7	32.3
Automated-fix review (51 events × 4 min)	0.0	3.4	-3.4
Malformed-commit triage (22 × 26 min)	9.5	0.0	9.5
Pipeline drift / bespoke debugging	6.0	1.5	4.5
Net monthly recovery	53.5	10.6	42.9

Table 1. Monthly operations effort, baseline versus steady state.

The framework returns roughly 42.9 operations hours per month — more than double the 20-hour target — with the OOM workstream alone accounting for 28.9 net hours. The figure is deliberately conservative: it charges four minutes of review against every automated remediation, assumes fifteen per cent of incidents still need a person, and excludes the premium cost of after-hours interruption.

Against an implementation cost of about 320 engineering hours over six sprints and Azure consumption charges near forty dollars per month, the framework pays back in under eight months at a fully loaded rate of ninety-five dollars per hour.

Onboarding Time to Under One Day

Onboarding takes three to five days today because a new engineer learns our conventions by reading four hundred lines of inherited YAML that differ in every repository, then discovers the unwritten rules through failed builds. The framework replaces that with a four-hour path: one hour on the golden-path document, one hour scaffolding a working pipeline from the CLI generator, one hour taking a first change through pre-commit

validation into a pull request, and one hour shadowing a deployment to development. The reduction is structural — twenty-five declarative lines instead of four hundred, with hooks teaching conventions at the moment of the mistake.

DORA Impact

DORA Metric	Baseline	Target (90 days)	Primary Driver
Deployment frequency	2–3 / week	Daily per service	Templates
Lead time for changes	3.5 days	< 8 hours	Templates
Change failure rate	22%	< 8%	Enforcement
Mean time to restore	47 minutes	< 4 minutes	Auto-remediation

Table 2. Projected DORA outcomes, ninety days after rollout.

Recommendation: approve a six-sprint build with two platform engineers — template library and warn-only enforcement first, auto-remediation in a pilot namespace next, then organisation-wide rollout with blocking enforcement.